Abstract Factory Design Pattern

Intent

Provide an interface for creating **families of related objects** without specifying their concrete classes — and guarantee that everything a given factory produces belongs to the **same family**.

Here the families are **HOME** and **OFFICE** furniture. If you pick the office factory, every product it hands back (chair *and* table) is office furniture. You physically cannot end up with a home chair next to an office table. That guarantee — *consistency across a product family* — is the whole reason Abstract Factory exists and is what separates it from a plain Factory Method.

UML class diagram

```
<<interface>> IChair <<interface>> ITable +sitOn() +use() ^ ^ ^ ^ | | | | HomeChair
OfficeChair HomeTable OfficeTable \_________|_______________|_________/ produced as
MATCHED FAMILIES by <<interface>> IFurnitureFactory +createChair() : IChair (no args!)
+createTable() : ITable ^ ^ | | HomeFurnitureFactory OfficeFurnitureFactory ^ ^ +-----
registered ---+ FurnitureFactoryProvider - factoryMap :
EnumMap<FurnitureType,IFurnitureFactory> + getFactory(FurnitureType)
```

The players

```
enums/FurnitureType the family key (HOME, OFFICE) product/IChair abstract product A
product/ITable abstract product B product/concret/HomeChair, OfficeChair concrete product
A, one per family product/concret/HomeTable, OfficeTable concrete product B, one per
family factory/IFurnitureFactory the ABSTRACT FACTORY — makes a chair AND a table
factory/concret/HomeFurnitureFactory concrete factory for the HOME family
factory/concret/OfficeFurnitureFactory concrete factory for the OFFICE family
factory/FurnitureFactoryProvider a registry that picks a factory by FurnitureType
```

Read the two dimensions carefully — this is the part people get wrong:

- **Product kind** — chair vs. table (the *rows*).
- **Family** — home vs. office (the *columns*).

An Abstract Factory has one concrete factory **per column**, and each produces one product **per row**. The grid:

	IChair	ITable
HomeFurnitureFactory	HomeChair	HomeTable
OfficeFurnitureFactory	OfficeChair	OfficeTable

The code, line by line

`FurnitureType` — the family key

```
public enum FurnitureType { HOME("HOME"), OFFICE("OFFICE"); private final String name; FurnitureType(final String name) { this.name = name; } @JsonValue public String getName() { return name; } }
```

- A **single** enum names the families. It is the one axis of variation. Chairs, tables, and factories are all keyed by this same enum, which keeps the model coherent.
- `private final String name` — final because an enum constant's label should never change. (An earlier draft used `public String name`, which both leaks the field and shadows the built-in `Enum.name()` method — fixed here.)
- `@JsonValue getName()` — API-friendly JSON value; not needed by the pattern.

`IChair` / `ITable` — the abstract products

```
public interface IChair { FurnitureType getFurnitureType(); void sitOn(); } public interface ITable { FurnitureType getFurnitureType(); void use(); }
```

- Each abstract product exposes its behavior (`sitOn()`, `use()`) plus `getFurnitureType()` so it can **declare which family it belongs to**. That self-declaration is what lets the provider index things automatically, exactly like the Factory module.

Concrete products

```
@Component public class HomeChair implements IChair { @Override public void sitOn() { System.out.println("Sitting on a home chair!"); } @Override public FurnitureType getFurnitureType() { return FurnitureType.HOME; } }
```

- `@Component` so Spring can inject them into the matching factory.
- `HomeChair`/`HomeTable` report `HOME`; `OfficeChair`/`OfficeTable` report `OFFICE`. Their family identity is baked in.

`IFurnitureFactory` — the abstract factory

```
public interface IFurnitureFactory { FurnitureType getFurnitureType(); IChair
createChair(); ITable createTable(); }
```

- This is the heart of the pattern. One factory creates a **whole family** — a chair *and* a table.
- **`createChair()` and `createTable()` take NO arguments.** This is deliberate and is the single most important design point (see *Why* below). The factory already knows its family; the caller does not get to influence it per call.
- `getFurnitureType()` lets the provider file each factory under its family.

Concrete factories

```
@Component @RequiredArgsConstructor public class HomeFurnitureFactory implements
IFurnitureFactory { private final HomeChair homeChair; private final HomeTable homeTable;
@Override public IChair createChair() { return homeChair; } @Override public ITable
createTable() { return homeTable; } @Override public FurnitureType getFurnitureType() {
return FurnitureType.HOME; } }
```

- `@RequiredArgsConstructor` (Lombok) generates a constructor for the two `final` fields, so Spring injects exactly the **home** products into the **home** factory.
- `createChair()` returns the home chair, `createTable()` returns the home table — a **matched set**. There is no way for this factory to ever return an office product. That is the guarantee.
- `OfficeFurnitureFactory` is the mirror image with `OfficeChair`/`OfficeTable`/`OFFICE`.

`FurnitureFactoryProvider` — the registry

```
@Component public class FurnitureFactoryProvider { private final EnumMap<FurnitureType,
IFurnitureFactory> factoryMap; public FurnitureFactoryProvider(List<? extends
IFurnitureFactory> factories) { this.factoryMap =
factories.stream().collect(Collectors.toMap( IFurnitureFactory::getFurnitureType,
Function.identity(), (a, b) -> b, () -> new EnumMap<>(FurnitureType.class))); } public
IFurnitureFactory getFactory(final FurnitureType furnitureType) { return
factoryMap.get(furnitureType); } }
```

- Structurally identical to `ShapeFactory` in the Factory module, but the values are **factories**, not products. Spring injects every `IFurnitureFactory`, and they get filed into an `EnumMap` by the family each reports.
- `getFactory(FurnitureType)` hands the caller the right concrete factory. From there, everything that factory makes is guaranteed consistent.

- This registry is a convenience layer (a factory-of-factories) so the client can select a family at runtime by enum. It is **not** part of the classic GoF diagram, but it fits this codebase's self-registering `EnumMap` convention.

The demo — `AbstractFactoryDesignPattern`

```
FurnitureFactoryProvider provider = context.getBean(FurnitureFactoryProvider.class);
IFurnitureFactory officeFactory = provider.getFactory(FurnitureType.OFFICE);
officeFactory.createChair().sitOn(); // "Sitting on an office chair!"
officeFactory.createTable().use(); // "Using an office table!" IFurnitureFactory
homeFactory = provider.getFactory(FurnitureType.HOME); homeFactory.createChair().sitOn();
// "Sitting on a home chair!" homeFactory.createTable().use(); // "Using a home table!"
```

You pick a family **once** (`getFactory(OFFICE)`), and every product after that is office furniture. That's the payoff.

Why the design decisions

Why do `createChair()` / `createTable()` take no arguments?

Because **the factory *is* the choice of family.** The moment you write `createChair(someQualityOrType)`, the caller — not the factory — is deciding what comes out, and the family guarantee evaporates. A factory that takes a per-call type argument is just a Factory Method wearing an Abstract Factory costume.

> This is exactly the bug the first version of this module had: `HomeFurnitureFactory` and `OfficeFurnitureFactory` were identical and both forwarded a `CHEAP`/`EXPENSIVE` argument to sub-factories, so HOME vs. OFFICE meant nothing. The fix was to make each concrete factory own its family and drop the arguments.

Why one family enum instead of two (quality + kind)?

The broken version had two orthogonal axes — HOME/OFFICE on the factories and CHEAP/EXPENSIVE on the products — that never lined up, so a "home" factory could emit an "expensive" chair and a "cheap" table with no coherence. Collapsing to **one** axis (family) is what makes a factory's output a genuine matched set. If you truly needed quality *and* location, that would be a **2-dimensional** Abstract Factory: four concrete factories (HomeCheap, HomeExpensive, OfficeCheap, OfficeExpensive), each still producing a fully consistent set.

Why inject the concrete products into each factory?

So the matched set is wired at construction time and cannot be violated. `HomeFurnitureFactory` literally only has a `HomeChair` and a `HomeTable` in its fields — the type system itself forbids it from returning office furniture.

Why the `EnumMap` provider?

Same reasoning as the Factory module: it lets a caller choose the family at runtime by enum, it auto-discovers new factories (add a new `IFurnitureFactory @Component` and it registers itself), and `EnumMap` is the fast, correct container for enum keys. Adding a THIRD family (say `GARDEN`) means: add `GARDEN` to the enum, add `GardenChair`/`GardenTable`, add `GardenFurnitureFactory` — and **nothing else changes**, including the provider.

Execution flow (as run from `main`)

```
AbstractFactoryDesignPattern.main ■ ■■■ SpringApplication.run(...) Spring discovers all
@Components ■ ■■■ HomeChair, HomeTable, OfficeChair, OfficeTable ■ ■■■
HomeFurnitureFactory ← injected HomeChair + HomeTable ■ ■■■ OfficeFurnitureFactory ←
injected OfficeChair + OfficeTable ■ ■■■ FurnitureFactoryProvider ← injected List[Home…,
Office…] ■ ■■■ EnumMap { HOME→HomeFurnitureFactory, OFFICE→OfficeFurnitureFactory } ■
■■■ provider.getFactory(OFFICE) → OfficeFurnitureFactory ■ ■■■ createChair() →
OfficeChair.sitOn() → "Sitting on an office chair!" ■ ■■■ createTable() →
OfficeTable.use() → "Using an office table!" ■ ■■■ provider.getFactory(HOME) →
HomeFurnitureFactory ■■■ createChair() → HomeChair.sitOn() → "Sitting on a home chair!"
■■■ createTable() → HomeTable.use() → "Using a home table!"
```

Factory Method vs. Abstract Factory (the one-line rule)

- **Factory Method** makes **one** product, chosen by a key you pass in — `getShape(TRIANGLE)`.
- **Abstract Factory** makes a **family** of products, and the *factory* is the choice — `getFactory(OFFICE).createChair()` / `.createTable()`.

If every `create...()` call needs a "type" argument, you have Factory Method. If picking the factory already determines everything it produces, you have Abstract Factory.

> Note: `SpringApplication.run(...)` boots the container so beans can be discovered and injected; `spring-boot-starter-web` keeps the process alive after the demo prints, so stop it with Ctrl-C. The pattern itself doesn't depend on Spring.